



COS 132 - Imperative Programming

Study Unit 9: Abstract Data Types

Copyright ©2024 by Linda Marshall, Inge Odendaal and Mark Botros. All rights reserved.

Contents

9.1	Introduction	2
9.2	Structures (structs) in C++	2
9.2.1	Defining structs in C++	4
9.2.2	Accessing the fields of a struct	5
9.3	Structures as Parameters	8
9.3.1	Pass-by-value	8
9.3.2	Pass-by-reference	9
9.4	Structures as Return Types	10
9.5	Pointers and structs	11
9.5.1	Pass-by-pointer	11
9.5.2	Return a pointer	12
9.5.3	New and Delete with Structs	12
9.5.4	Dynamic arrays as member fields of structs	13
9.6	Advantageous and Disadvantageous of using structs	14
9.7	Acknowledgements	14
9.8	Document Change Log	14

9.1 Introduction

In this study unit we will consider the structure (**struct**) Abstract Data Type (ADT) defined in C++. In later modules, the object-oriented programming paradigm will be introduced and used where **classes** will be discussed. Classes are considered ADTs that promote encapsulation of the members of the class.

The sections that follow will discuss how structures in C++ can be defined and used. Much of the study unit is concerned with using **structs** with functions and arrays. When considering functions, structures can be passed as parameters or returned by the function. In both these cases, either a copy of the structure is made when passing it or a reference to an existing structure to be passed to or from the function. It is not necessary to only define structures on the stack, they can easily be placed on the heap by using pointers.

9.2 Structures (structs) in C++

Structures in C++ are useful to combine items of possibly different data types together and have a single name when referring to them. Consider the following program that reads in the number of students for which marks must be entered. For each student a name and five subject marks (**mark1** through to **mark5**) are entered. The average of these five marks is calculated and stored.

With the knowledge we have till now, we can either create multiple static or dynamic arrays. In this solution, we will create 7 arrays of the same size (or 2 arrays and a matrix of size $n \times 5$, where **n** is the number of students, to capture the marks. Listing 9.1, given below, illustrates the solution using 7 arrays. In this code, it is assumed that there will be no more than **MaxStudent** items captured and that the user will enter a value of the actual number of students to be catered for, thereby wasting valuable stack space due to part of each array potentially not being unused.

```
#include <iostream>
#include <string>

using namespace std;

const int MaxStudent = 50;

int main() {
    string name[MaxStudent];
    int mark1[MaxStudent];
    int mark2[MaxStudent];
    int mark3[MaxStudent];
    int mark4[MaxStudent];
    int mark5[MaxStudent];
    double average[MaxStudent];
    int studNum;

    cout << "Please enter the number of students: ";
    cin >> studNum;
```

```

if ((studNum > 0) && (studNum <= MaxStudent)) {
    for (int i = 0; i < StudNum; i++) {
        cout << "Name: ";
        cin >> name[i];
        cout << "First_Mark: ";
        cin >> mark1[i];
        cout << "Second_Mark: ";
        cin >> mark2[i];
        cout << "Third_Mark: ";
        cin >> mark3[i];
        cout << "Fourth_Mark: ";
        cin >> mark4[i];
        cout << "Fifth_Mark: ";
        cin >> mark5[i];

        average[i] = (mark1[i] + mark2[i] + mark3[i] + mark4[i] + mark5[i]) / 5;
    }

    /*
    Determine the average of all the averages
    Sort the name according to averages
    Determine the name of the student who performed closest to the average
    */

    //Output used to see if code works.
    for (int i = 0; i < studNum; i++) {
        cout << "Student: " << i << endl;
        cout << "\tName: " << name[i] << endl;
        cout << "\tFirst_Mark: " << mark1[i] << endl;
        cout << "\tSecond_Mark: " << mark2[i] << endl;
        cout << "\tThird_Mark: " << mark3[i] << endl;
        cout << "\tFourth_Mark: " << mark4[i] << endl;
        cout << "\tFifth_Mark: " << mark5[i] << endl;
        cout << "\tAverage: " << average[i] << endl;
        cout << endl;
    }
}

```

Listing 9.1: Entering the Marks of Multiple Students using Arrays

Question 9.1

Write the sections of code that has been specified in the comments (*/* ... */*) in Listing 9.1.

This program becomes clumsy as each student has a name, five marks and an average. When the student information is sorted a function that accepts the **name**, **mark1** through to **mark5** and **average** arrays must be written. The swapping part of the function must exchange the i^{th} and j^{th} elements in each of the arrays to keep the data valid.

The next section will introduce a structure that can be used to keep the information of a single student, that is the name, five marks and the average, together. This structure can be placed in an array and manipulated as a single unit.

9.2.1 Defining structs in C++

A structure in C++ is defined using the **struct** keyword. Structures need to be defined before they are used for the first time in a program. Structures comprise of fields. A field is either a member variable or a member function.

A **struct** is defined as a collection of values assigned to fields. You may recall an array was also defined as a collection of values, but **structs** have two unique properties:

- The stored values can **have different types**, i.e., they are heterogeneous while in arrays all elements are of the same type, and therefore homogeneous
- The stored values are **called fields or members and are given identifiers**. A field is individually named like ordinary variables, unlike arrays where components are selected according to their position (index) in the array.

It is good practice to define a structure in its own header file along with functions that make use of or manipulate the structure. Below is an example of such a header file.

```
#ifndef STRUCTOUTLINE_H
#define STRUCTOUTLINE_H

struct StructName {
    //field or member variables
    Type1 fieldName1;
    ...
    TypeN fieldNameN;

    //field or member functions
    ReturnType1 function1(parameterlist);
    ...
    ReturnTypeN functionN(parameterlist);
};

// External functions that manipulate the struct.

#endif /*STRUCTOUTLINE_H*/
```

Listing 9.2: Generic Definition of a **struct**

In this study unit, we will only consider the member variable type fields and not the member functions. We will make use of functions external to the **struct** to provide behaviour for the structure.

To solve the problems associated with the program given in Listing 9.1, a structure is defined that contains the name, five marks and average for a (one) student. An array of this structure is then defined.

```

#ifndef STUDENT_H
#define STUDENT_H

#include <string>

struct Student {
    std::string name;
    int mark1;
    int mark2;
    int mark3;
    int mark4;
    int mark5;
    double average;
};

#endif /*STUDENT_H*/

```

Listing 9.3: First Attempt at a **Student** structure

The array is defined in the following manner:

```
Student studArray[MaxStudent];
```

The way in which the **Student** type is defined is not the most efficient method of defining the name, marks and average of the student. The **field_type** of a field does not need to be a simple type, but may itself be a complex type. This means that all the marks may be joined in one complex structure - such as an array.

```

struct Student {
    std::string name;
    int marks[5];
    double average;
};

```

Listing 9.4: Final Definition of the **Student** structure

Sorting of an array of structures is so much easier as it only requires the swopping of values of this structure. The parameters of the sort are therefore simplified to a single array holding values of type **StructName**, in our case of type **Student**.

9.2.2 Accessing the fields of a struct

To access a field in a **struct**, a variable of the specific **struct** type must be defined and the individual fields are accessed using the dot-notation (**.**), also referred to as direct field access. The listing below provides the ways to access the fields of a structure.

We use the dot-notation along with the assignment operator to set and get field values. Listing 9.5 provides an overview of how this is done using the generic **struct** definition given in Listing 9.2.

```

int main() {
    StructName structVariable;

```

```

    /*
    Setting of member variables in the struct
    */
    structVariable.fieldName1 = ...;
    ...
    structVariable.fieldNameN = ...;

    /*
    Getting members variables of the struct
    */
    fieldType1 v1 = structVariable.fieldName1;
    ...
    fieldTypeN vN = structVariable.fieldName1;

    return 0;
}

```

Listing 9.5: Setting and Getting **struct** field values

The following program illustrates how the fields of the `Student` structure in Listing 9.4 – defined in the header file `Student.h` – are accessed using the dot-notation. Note how the `Student[0]` array element is assigned using an additional variable (`MyStudent`) of type `Student` compared to `Student[1]`.

```

#include <iostream>

#include "Student.h"

using namespace std;

const int MaxStudent = 50;

int main() {
    Student student[MaxStudent];
    Student myStudent;
    int studNum = 2;

    myStudent.name = "John_Smith";
    myStudent.marks[0] = 40;
    myStudent.marks[1] = 50;
    myStudent.marks[2] = 60;
    myStudent.marks[3] = 70;
    myStudent.marks[4] = 80;

    student[0] = myStudent;

    student[1].name = "Paul_Jones";
    student[1].marks[0] = 90;
    student[1].marks[1] = 80;
    student[1].marks[2] = 50;
}

```

```

    student[1].marks[3] = 70;
    student[1].marks[4] = 80;

    ...
}

```

Listing 9.6: Assigning values to **Student**

Question 9.2

How can the marks of the i^{th} student (of type **Student**) be accessed?

The program in Listing 9.1 can now be written as follows (refer to Listing 9.7). This program is easier to read and understand.

```

1  #include <iostream>
2  #include <string>
3
4  #include "Student.h"
5
6  using namespace std;
7
8  const int MaxStudent = 50;
9
10 int main() {
11     Student studArray[MaxStudent];
12     int studNum;
13
14     cout << "Please enter the number of students:\n";
15     cin >> studNum;
16
17     if ((studNum > 0) && (studNum <= MaxStudent)) {
18         for (int i = 0; i < studNum; i++) {
19             cout << "Name:\n";
20             cin >> studArray[i].name;
21             double sum = 0;
22             for(int j=0; j < 5; j++){
23                 cout << "Mark_" << (j+1) << ":\n";
24                 cin >> studArray[i].marks[j];
25                 sum += studArray[i].marks[j];
26             }
27
28             studArray[i].average = sum / 5.0;
29         }
30
31         /*
32         Determine the average of all the averages
33         Sort the name according to averages
34         Determine the name of the student who
35         performed closest to the average
36         */
37

```

```

38      //Output used to see if code works.
39      for (int i = 0; i < studNum; i++) {
40          cout << "Student:_" << i << endl;
41          cout << "\tName:_" << studArray[i].name << endl;
42          for (int j = 0; j < 5; j++) {
43              cout >> "\tMark_" << j << ":_" << studArray[i].mark[j] << endl;
44          }
45          cout << "\tAverage:_" << studArray[i].average << endl;
46          cout << endl;
47      }
48  }
49  }

```

Listing 9.7: Entering the Marks using an Array of `structs`

Question 9.3

Write a function to sort the student array. Send the entire student array (`studArray`) of `Student` type as a parameter to the sorting function. You may assume you are sorting on the `name` field of the `Student` type.

9.3 Structures as Parameters

Structures may be used in parameter lists as either pass-by-value or pass-by-reference parameters. To illustrate this, consider the following function definitions.

```

void printStudent (Student student);
void populateStudent(Student &student);
void getStudent (string name, Student arr[], int size, Student &student);

```

Listing 9.8: Functions illustrating sending structures as parameters

The first function definition (`printStudent`) is an example of parameter passing by value. This means the function cannot change the values associated with the structure fields. `populateStudent` and `getStudent` make use of pass-by-reference.

9.3.1 Pass-by-value

The function `printStudent`, given in Listing 9.8 takes as parameter a single `Student` value. The implementation of this function is given in the listing below. Any changes to the values of the fields of the structure sent to a function as a pass-by-value parameter within the function does not impact on the values of the structure in the calling function.

```

void printStudent(Student student) {
    cout << "Name:_" << student.name << endl;
    for (int j = 0; j < 5; j++) {
        cout << "Mark_" << (j + 1) << ":_" << student.marks[j] << endl;
    }
    cout << "Average:_" << student.average << endl;
    cout << endl;
}

```



```
}
```

Listing 9.9: Pass-by-value `printStudent` Function

9.3.2 Pass-by-reference

The parameter of the `printStudent` function can be defined using pass-by-reference. The definition of the function changes to the following:

```
void printStudent(const Student &student);
```

Applying `const` to the structure when passing by reference is equivalent to passing by value. In this case, values cannot be assigned to variables of the structure within the function. Doing so will result in a compile-time error. The implementation of the function remains the same as given in Listing 9.9, only the definition changes.

The function `populateStudent` in Listing 9.8 asks the user to enter the values for the fields of the `Student` structure. These values are passed back to the calling function. The implementation of the function is given in the following code:

```
void populateStudent(Student &student) {
    cout << "Name: ";
    cin >> student.name;
    double sum = 0;
    for (int j = 0; j < 5; j++) {
        cout << "Mark_" << (j + 1) << ": ";
        cin >> student.marks[j];
        sum += student.marks[j];
    }

    student.average = sum / 5.0;
}
```

The following call to the function replaces the code in Lines 19 to 29 of the program given in Listing 9.7 with

```
populateStudent(studArray[i]);
```

The `getStudent` function in Listing 9.8, implements a linear search for a student based on their name in an array (`arr`) of `Student` elements and returns a reference to the student, via the `student` parameter if the student entry exists. If the student entry does not exist, all field values of the `struct` will receive default values. An implementation of the function is given by:

```
void getStudent (string name, Student arr[], int size, Student &student){
    int i = 0;
    while ((arr[i].name != name) && (i < size)) {
        i++;
    }
    if (i == size) {
        // name was not found - create the default student
        student.name = "";
    }
}
```

```

        for (int i = 0; i < 5; i++)
            student.marks[i] = 0;
        student.average = 0;
    } else {
        // name was found – assign the corresponding array element to student
        student = arr[i];
    }
}

```

Listing 9.10: `getStudent` function - pass-by-reference

9.4 Structures as Return Types

Structures can be returned to the calling function as a return type. The return can be by value, where a copy of the structure is made, by reference or as a pointer to the element in the original array.

The function `getStudent` can be updated to return a copy of a student using the return type rather than as a pass-by-reference parameter. The function header is updated to the following:

```
Student getStudent(string name, Student arr[], int size);
```

Question 9.4

Write the updated `getStudent` function with function header that returns a `Student` type by making use of the function defined in Listing 9.10.

The function for `getStudent` (refer to Listing 9.10 and your answer to Question 9.4 includes code to set all the values of the structure to default values. Rather than repeating this code in multiple locations, we can write a function that returns a default student structure. The definition for the structure is given by:

```
Student createDefaultStudent();
```

Question 9.5

Implement the `createDefaultStudent` function.

Question 9.6

Replace the code in your updated `getStudent` function of Question 9.4 with a call to the `createDefaultStudent` function to create the default student.

Functions can also return a `struct` by reference. Consider the implementation for the `increaseStudentMarks` function given below. This function accepts as a parameter the student structure for whom the marks are to be increased and a floating point value used to scale the marks accordingly. The function returns the an updated version of the student structure without changing the values of the original. This leaves how the updated student structure is to be used to the discretion of the programmer.

```
Student& increaseStudentMarks(Student &student, float markScale) {
    Student s;
    float sum = 0;
    s.name = student.name;

```

```

for(int i=0; i < 5; i++) {
    s.marks[i] += student.marks[i] * markScale;
    sum += student.marks[i];
}
s.average = sum/5.0;
return s;
}

```

9.5 Pointers and structs

As with other types, we can define a pointer to a structure. To access the fields of the structure we no longer make use of the dot-notation (.). We need to use the arrow (->) notation (also referred to as field access or structure pointer dereference notation) as the fields are contained in a structure on the heap.

As has been discussed earlier, we can pass an instance of a **struct** as either a value or a reference to a function as a parameter. We can also return a **struct** as either a value or a reference from a function. In this section, we will discuss how an instance of a **struct** can be passed as a pointer into a function or returned from a function. Lastly this section will also discuss the allocation and deallocation of **structs** to/from heap memory using **new** and **delete** respectively.

9.5.1 Pass-by-pointer

In order to pass an instance of a **struct** as a pointer into a function, the following parameter type should be used:

```

/*In the appropriate file declare the struct.*/
struct StructName{
/*Struct interior goes here*/
};

```

```

ReturnType functionName(...,StructName *parameterName,...);
/*Note the ... refers to other parameters being specified*/

```

An example of a function that takes in a pointer to an instantiation of a **struct** is given below:

```

void printStudent(Student *student)

```

As can be seen, the parameter is a pointer to the type Student. The code example below provides an example of how the fields of the passed-in parameter can be accessed using the arrow (->) operator.

```

void printStudent(Student *student) {
    cout << "Name:_" << student->name << endl;
    for (int j = 0; j < 5; j++) {
        cout << "Mark_" << (j + 1) << ":_:" << student->marks[j] << endl;
    }
}

```

```

    cout << "Average:_" << student->average << endl;
    cout << endl;
}

```

Question 9.7

Implement the `populateStudent` function using the following function signature:

```
void populateStudent(Student *student)
```

This function should ask the user for the student's name, and the 5 required marks. The function should also calculate the average of the student's marks.

9.5.2 Return a pointer

We saw that we can return **structs** by value and by reference, but it is also possible to return an instantiation of a **struct** from a function.

This is accomplished using the following function signature outline:

```
StructName* functionName(/*Parameters goes here*/)
```

As can be seen the return type is a pointer of the type **StructName**. A concrete example is given below:

```
Student* createDefaultStudent();
```

Even though the **new** operator is used within the `createDefaultStudent`, the onus is on the function receiving the newly created **Student** structure to delete it.

Question 9.8

Explain why dynamic memory needs to be used in the implementation of this function.

9.5.3 New and Delete with Structs

Just as with dynamically allocated arrays, **structs** can also be dynamically allocated. This means that instead of the **struct** being allocated on the static stack memory, the **struct** is allocated on the dynamic heap memory. This is accomplished by using the **new** and **delete** operators. The code below illustrates the recipe of how to allocate and deallocate a **struct**:

```

StructType* variableName = new StructType;
...
delete variableName;

```

There are a few pitfalls that need to be taken into account when dynamically allocating **structs**. These pitfalls are very similar to those when processing dynamically allocated arrays. If you are unsure about these pitfalls please revise the previous section on memory management. Note, memory management becomes very important in follow-up modules, especially COS 110 and COS 214.

9.5.4 Dynamic arrays as member fields of structs

Consider we change the `Student` `struct` we have been working with so far to the following structure:

```
struct Student{
    string name;
    int* marks;
    int numMarks;
    double average;
};
```

In this case, the `marks` member variable is a dynamic array which needs to be deallocated before the instance of the `Student` is deallocated/goes out of scope. This can be done in the same way as a standard dynamic array is deallocated, as illustrated below:

```
void someFunc(){
    Student student;
    student.numMarks = 10;
    student.marks = new int[numMarks];
    /*
    more code
    */
    delete [] student.marks;
}
```

In this example, the dynamic array of the student instance is allocated and possibly used and then finally deallocated before the student instance goes out of scope.

Given a class `struct` as follows:

```
struct Classroom {
    Student **students; /*dynamic 1D array of dynamic Student objects*/
    int numberOfStudents;
};
```

```
ClassRoom* createClass();
```

The `createClass` function can possibly be implemented as follows:

```
ClassRoom* createClass(){
    Classroom *returnVariable = new Classroom;

    cout << "How big is the class?";
    cin >> returnVariable->numberOfStudents;
    if(returnVariable->numberOfStudents < 0){
        return NULL;
    }
    returnVariable->students = new Student*[returnVariable->numberOfStudents];
    for(int i=0; i < returnVariable->numberOfStudents; i++){
        returnVariable->students[i] = new Student;
    }
    /*
```

```

        populate the i-th student in the dynamic array.
        */
    }

    return returnVariable;
}

```

As can be seen, we are able to create a dynamic `ClassRoom` instance, which contains a dynamic array of dynamic `Student` instances.

Question 9.9

Using the `ClassRoom` struct above, implement the following function:

```
void deallocateClass(ClassRoom *classRoom);
```

This function should deallocate all of the dynamic memory allocated in the `ClassRoom` structure instance as well as the dynamic memory that was allocated to the member variables.

9.6 Advantageous and Disadvantageous of using structs

Structures are a powerful concept and help to bring together variables of different types in a single user-named type. As was seen when the original student management program was translated to a program using an array of `Student` types, the complexity of the program decreased.

A major disadvantage of `structs` is that by default the field are accessible to any function using the structure. Another disadvantage is that standard operators are not available for types defined using `structs`. That is, you cannot apply operators such as the mathematical or relational operators to variables of `structs`. Classes, which will be covered in detail when Object Orientation is presented during the next semester, will help to solve some of these problems.

9.7 Acknowledgements

Special thank you to Cobus Redelinghuys for the translations of the C++ examples and providing additional examples and accompanying text for these examples.

9.8 Document Change Log

Date	Change
19 June 2024	Initial preparation and presentation of the document.